GEO Monitoring Service

Full services architecture on AWS (target state)

Document	Version	Status	Owner	Date
Services architecture	1.0	Draft	Rui, JiLai AI Tech	12 Oct 2026

1. Purpose and scope

This document describes the target architecture of the GEO (Generative Engine Optimization) monitoring service built by JiLai AI Tech. The service measures how often and how accurately a brand appears in answers from AI engines such as ChatGPT, Claude, Perplexity and Gemini, tracks it month by month, and turns the gaps into concrete tasks for a marketing team.

It covers the logical components, the AWS deployment, data flows, data model, security and GDPR, scaling, cost drivers and the path from the first pilot version to this target. The pilot version is described in the separate document *GEO Monitoring Service - First Version Architecture*.

2. Business context

The service is delivered through marketing agencies. The agency's team runs the marketing work for its clients; JiLai runs the AI measurement and intelligence behind it as a hosted service.

Party	Role	What they get
Agency (tenant)	Sells GEO services to its own clients under its brand	White-label dashboard, monthly reports, action plans per client
Agency client (brand)	The company being monitored	Visibility status, wrong facts found, progress over time
JiLai AI Tech	Builds and operates the platform	Keeps code, prompt libraries and scoring logic; per-client setup and monthly fees

Design principle: **teach the method, keep the machine**. Agencies use the dashboard and reports; prompts, scoring and code stay server-side.

3. Logical architecture

The core is a deterministic batch pipeline, not an agent system. Results must be reproducible and comparable month to month, and cheap to run per client. LLMs are used only to draft prompt lists at onboarding and to judge answers in the analyzer.

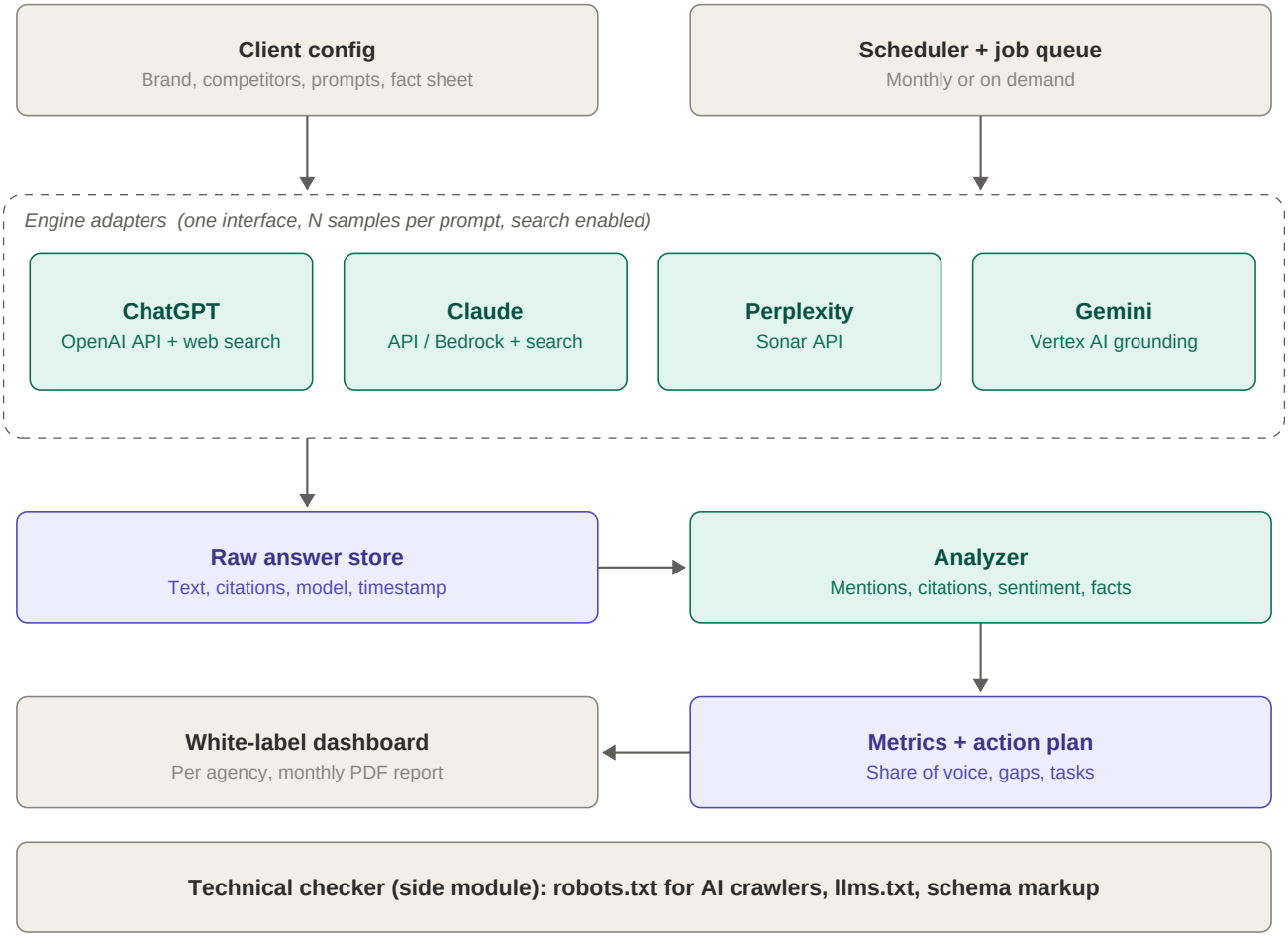

Figure 1. Logical pipeline

3.1 Components

Component	Responsibility	Notes
Client config	Brand name and aliases, domain, 3-5 competitors, markets and languages (ES/EN), brand fact sheet, 30-50 prompts tagged by topic and funnel stage	Prompt generator crawls the client site and drafts buyer questions; agency approves them
Scheduler + queue	Starts monthly runs and on-demand runs; one job per client	EventBridge Scheduler + SQS with dead-letter queue
Engine adapters	Common interface run(prompt) -> answer, cited URLs, model, timestamp	Search-enabled modes only; each prompt sampled 3x; rate limiting and retries per engine
Raw answer store	Every raw answer kept permanently	Basis for cross-client industry benchmarks
Analyzer	Brand/competitor mentions and position, cited URL classification, sentiment, fact check against fact sheet	Alias matching first; small LLM judge for ambiguous cases
Metrics + action plan	Mention rate, share of voice, citation share, trends; gaps converted into tasks	e.g. directories to get listed in, pages to write, wrong facts to fix at source
Dashboard + reports	Multi-tenant UI: agency -> clients -> runs; monthly PDF	Agency branding; read-only for agency users
Technical checker	robots.txt rules for AI crawlers, llms.txt, schema markup	Runs with each audit

4. AWS deployment architecture

Target deployment in an EU region (default eu-south-2, Spain; eu-west-1 or eu-central-1 if a required service is missing there). EC2 is the compute layer for both the web tier and the workers.

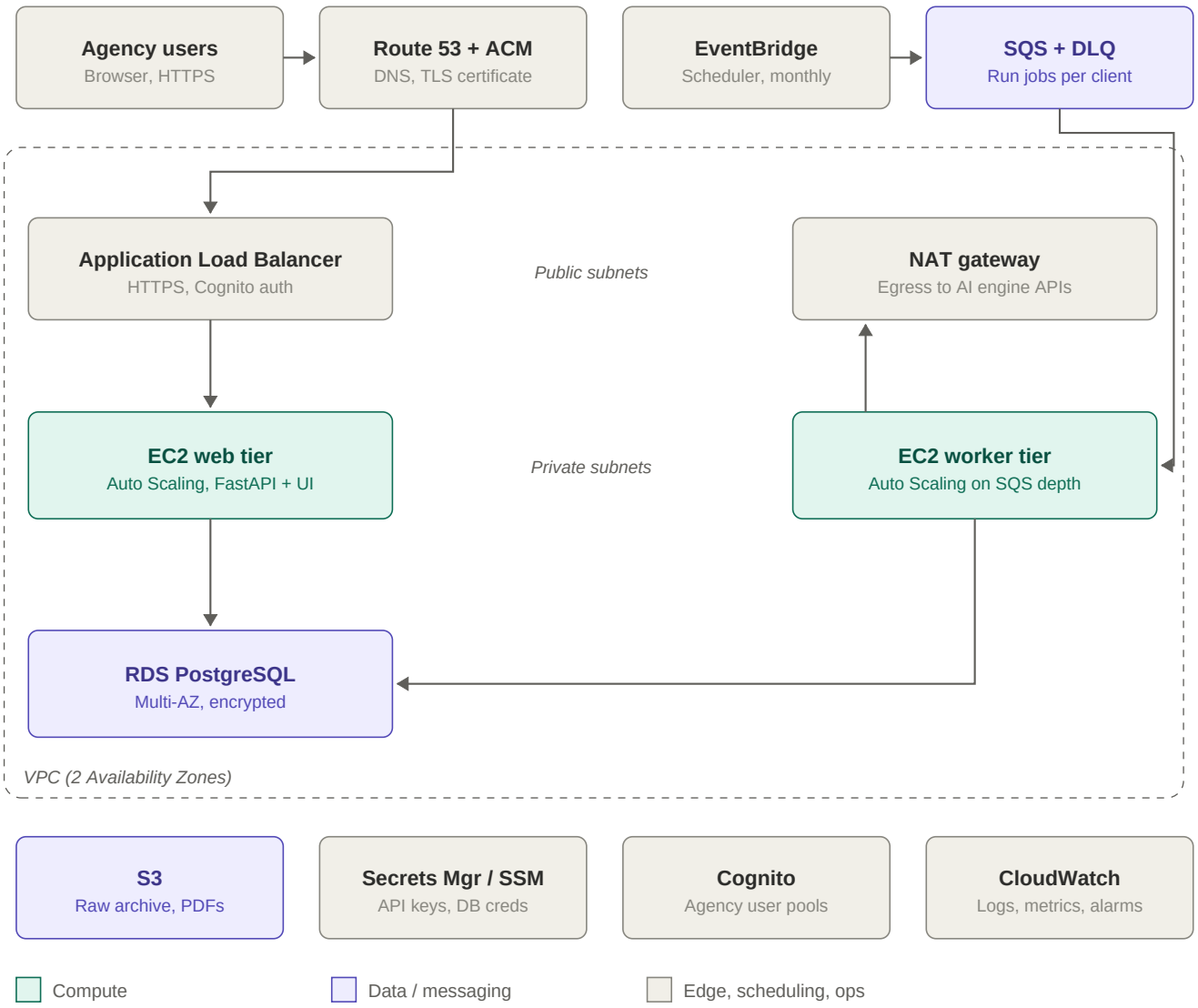

Figure 2. AWS deployment, target state

4.1 AWS services

Service	Use	Configuration
Route 53 + ACM	DNS for the dashboard domain; TLS certificate	Per-agency custom domains optional
Application Load Balancer	HTTPS entry point; routes to web tier	Cognito authentication action on listener
EC2 web tier	FastAPI backend and dashboard	Auto Scaling group, 2 instances across 2 AZs, Graviton
EC2 worker tier	Consumes SQS jobs, calls AI engines, analyzes, writes results	Auto Scaling 0-N on queue depth; scales to zero between runs
SQS + DLQ	Job queue; failed jobs after 3 attempts go to DLQ	Long polling, visibility timeout extended by worker
EventBridge Scheduler	Monthly run trigger	Cron in Europe/Madrid time zone

Service	Use	Configuration
NAT gateway	Outbound internet for private subnets (AI engine APIs)	One per AZ for full HA, or one to save cost
RDS PostgreSQL	Configs, runs, answers, metrics, tasks	Multi-AZ, encrypted, 7-30 day backups
S3	Raw answer archive, PDF reports, exports	Private, encrypted, lifecycle to Glacier IR after 180 days
Secrets Manager / SSM	LLM API keys, DB credentials	Read at runtime via instance role
Cognito	Agency user login, one group per tenant	MFA optional
CloudWatch	Logs, metrics, alarms (failed runs, DLQ depth, cost)	Plus AWS Budgets alert
Amazon Bedrock (optional)	Claude in an EU region under the AWS DPA	Check web search support vs direct API before use

5. Data flows

5.1 Monthly monitoring run

- EventBridge Scheduler sends `{"type": "monthly_run"}` to SQS.
- A worker expands it into one `client_run` message per active client.
- For each client job, the worker loads the config from RDS and sends every prompt to every enabled engine, 3 samples each, through the NAT gateway.
- Raw answers are written to RDS (JSONB) and archived to S3.
- The analyzer extracts mentions, citations, sentiment and fact errors; metrics and the action plan are computed and stored.
- A PDF report is generated to S3; the agency sees the new run on the dashboard. The SQS message is deleted.

5.2 Dashboard request

- Agency user signs in through Cognito at the ALB.
- The web tier reads only that tenant's clients, runs and metrics from RDS (tenant ID enforced on every query).
- On-demand runs from the dashboard send a `client_run` message to SQS.

6. Data model (core tables)

Table	Key fields
tenants	id, name, branding (logo, colors), plan
clients	id, tenant_id, brand, aliases, domain, markets, languages, fact_sheet, active
competitors	id, client_id, name, aliases, domain
prompts	id, client_id, text, language, topic, funnel_stage, active
runs	id, client_id, started_at, finished_at, status, engines, samples
answers	id, run_id, prompt_id, engine, model, sample_no, text, cited_urls (JSONB), raw (JSONB)

Table	Key fields
mentions	id, answer_id, entity (brand/competitor), position, sentiment, description
citations	id, answer_id, url, domain, source_type (own/competitor/directory/media/forum)
fact_issues	id, answer_id, claim, expected, severity
metrics	id, run_id, engine, topic, mention_rate, share_of_voice, citation_share
tasks	id, run_id, client_id, type, title, detail, priority, status

7. Security and GDPR

- All data stays in an EU AWS region; S3, RDS, SQS and EBS are encrypted at rest; TLS everywhere.
- Prompts and answers contain brand and market data, not personal data. Agency user accounts are the only personal data (Cognito). Data processing agreements are signed with AWS and each AI provider.
- Gemini via Vertex AI in an EU region; Claude optionally via Bedrock in an EU region.
- No SSH: instance access through SSM Session Manager. IMDSv2 required. Least-privilege IAM roles.
- Secrets only in Secrets Manager / SSM, never in code, images, Terraform state or env files.
- Strict tenant isolation in the application layer; every query filtered by tenant_id.
- Agency contract: licence to use dashboard and reports; code, prompt libraries and methods remain JiLai's.

8. Scaling and reliability

- Workers scale on SQS queue depth and back to zero; monthly batches cost compute only while running.
- Per-engine rate limits and exponential backoff; a failing engine does not fail the whole client run.
- Idempotent jobs: a retried client_run overwrites its own partial results.
- Web tier on two AZs behind the ALB; RDS Multi-AZ with automated backups.
- Alarms: DLQ not empty, run duration over threshold, error rate per engine, budget forecast.

9. Cost drivers

Driver	Order of magnitude	How to control
AI engine API calls	50 prompts x 4 engines x 3 samples = about 600 calls per client per month	Cheaper models for analysis; cache prompt generation; tune samples per engine
Always-on infrastructure	ALB, NAT gateway, 2 web instances, RDS Multi-AZ	Only add when client count justifies it (see first version)
Worker compute	Hours per month	Scale to zero; Graviton instances
Storage	Small (GB range)	S3 lifecycle rules

Use the AWS Pricing Calculator for current figures in the chosen region.

10. Evolution from the first version

Stage	Trigger	Change
V1 pilot	First 1-3 clients	Single EC2, RDS single-AZ, SQS, Scheduler, S3 (see V1 document)
V2	Second agency or ~10 clients	Split worker into its own Auto Scaling group; Cognito login
V3 (this target)	Paying agencies, SLA expectations	ALB, private subnets + NAT, web tier on 2 AZs, RDS Multi-AZ
Later	Enough history	Google AI Overviews via SERP provider, industry benchmarks, alerts

The application code does not change between stages: workers read from SQS from day one.